

Machine Learning Intrusion Detection System (IDS)

Practical Demonstration Flow & Comprehensive Project Documentation

Topic: Intrusion Detection using ML	Algorithm: Random Forest Classifier	Dataset: NSL-KDD Benchmark
Classification: Binary (Normal vs Attack)	Accuracy: 99.77% on Test Data	Stack: Python, Flask, Scikit-learn

1. Sample Demonstration Flow (For Your Practical / Viva)

Follow this exact step-by-step procedure during your practical evaluation to demonstrate the working system effortlessly to the examiner:

• Step 1: Start System & Open Dashboard

Run `python app.py` in terminal and navigate to **http://127.0.0.1:5000**. Show the Home Dashboard displaying model cards: Random Forest, NSL-KDD, Binary Classification, and the dynamically loaded **99.77% accuracy**. Point out the live traffic donut chart and top feature importance graph.

• Step 2: Navigate to 'Detect Intrusion'

Click **Detect Intrusion** on the top navigation bar. Point out the two demonstration modes: **Mode B (1-Click Traffic Presets)** for instant evaluation and **Mode A (Manual Feature Input)** with 13 network traffic features.

• Step 3: Test Legitimate Network Traffic (NORMAL)

Click the green button: **[Normal Traffic Example]**. The form instantly populates legitimate HTTP connection features (protocol: tcp, service: http, flag: SF, src_bytes: 232, dst_bytes: 8153, count: 5). Click **PREDICT ACTIVITY**.

Result Displayed: Large Green Banner: **NORMAL ACTIVITY** | Confidence: **92.34%** | Risk: **LOW**. Explain the disclaimer confirming this is a machine learning statistical prediction.

• Step 4: Test Malicious Attack Traffic (INTRUSION - DoS)

Click the red button: **[Attack Traffic Example]**. The form auto-populates a classic Neptune SYN flood attack (protocol: tcp, service: private, flag: S0 (unanswered SYN), count: 123 (high connection burst), src_bytes: 0). Click **PREDICT ACTIVITY**.

Result Displayed: Large Crimson Banner: **INTRUSION DETECTED** | Confidence: **100.0%** | Risk: **HIGH**. The explanation notes abnormal SYN flags and high connection bursts.

• Step 5: Test Reconnaissance Attack (Port Sweep Probe)

Click the yellow button: **[Port Scan Attack]**. Populates port scan recon traffic (flag: REJ, diff_srv_rate: 1.0). Click **PREDICT ACTIVITY** → Accurately flagged as **INTRUSION** (HIGH Risk).

• Step 6: Show Actual Model Performance & Confusion Matrix

Navigate to **Model Performance**. Show the examiner the genuine 2x2 Confusion Matrix and Classification Report generated from the 10,000 held-out test records.

2. Project Overview & System Architecture

Problem Statement: Traditional signature-based IDS fail against mutated and zero-day attacks. This project implements a machine learning anomaly detection system that identifies malicious network flows using statistical traffic features.

Machine Learning Pipeline:

Dataset (NSL-KDD) → **Data Cleaning & Labeling** (normal = NORMAL, attacks = INTRUSION) → **Feature Selection** (13 connection & traffic features) → **Preprocessing** (OneHotEncoder + StandardScaler) → **80/20 Train/Test Split** → **Random Forest Classifier** (100 Trees) → **Evaluation** → **Model Serialization** (joblib) → **Flask REST API** → **Modern Web Interface**.

13 High-Impact Features Selected from NSL-KDD:

Category	Feature Name	Description & Cybersecurity Significance
Protocol & Flags	protocol_type, service, flag	Protocol (TCP/UDP/ICMP), network service port, and TCP status flag (SF = normal handshake, S0 = SYN flood, REJ = rejected connection).
Payload & Session	duration, src_bytes, dst_bytes, logged_in	Duration in seconds, bytes transmitted each way, and authentication flag (1 = successfully logged in, 0 = guest/unauthenticated).
Traffic Frequencies	count, srv_count	Connections to same host or service in past 2-second window. Massive bursts indicate DoS attacks.
Traffic Ratios	same_srv_rate, diff_srv_rate	% connections to same vs different services. High diff_srv_rate strongly indicates port scanning / probing.
Host-Based Stats	dst_host_srv_count, dst_host_same_srv_rate	Destination host service connection density and uniformity over multiple sessions.

3. Model Evaluation Results & Confusion Matrix

The Random Forest model was evaluated on **10,000 held-out test samples** from the NSL-KDD benchmark. All metrics below are genuine and dynamically recorded in results/model_metrics.json:

Metric	Score	Mathematical Formula	Cybersecurity Practical Meaning
Accuracy	99.77%	$(TP + TN) / \text{Total}$	Overall correct classifications across all test traffic.
Precision	99.77%	$TP / (TP + FP)$	Minimizes false alarms (legitimate traffic mistakenly blocked).
Recall	99.77%	$TP / (TP + FN)$	Critical: Minimizes missed attacks (intrusions slipping through).
F1-Score	99.77%	$2 \times (P \times R) / (P + R)$	Harmonic balance between precision and recall.

Actual Evaluated Confusion Matrix (10,000 Test Records):

	Predicted NORMAL	Predicted INTRUSION	Class Analysis
Actual NORMAL	5,308 (TN) True Negatives	3 (FP) False Alarms (Type I)	Normal traffic correctly identified with 99.94% recall.
Actual INTRUSION	20 (FN) Missed Attacks (Type II)	4,669 (TP) Accurate Detection	4,669 attacks intercepted out of 4,689 total (99.57% detection rate).

Top Features Influencing Random Forest Splits (Gini Importance):

Rank	Feature Name	Weight	Why it's important in Cyber Detection
#1	src_bytes	13.61%	Payload size sent. Floods often send 0 bytes or anomalous fixed packet sizes.
#2	dst_bytes	12.89%	Payload received. Benign sessions receive large data; DoS/probes receive almost none.
#3	flag_SF	8.87%	Normal SYN-ACK handshake completion flag; absence indicates probe or flood.
#4	same_srv_rate	7.65%	Density of connections targeted at a single service.
#5	diff_srv_rate	7.31%	Scans and probes target multiple diverse ports/services consecutively.
#6	count	6.12%	Rapid connection burst within 2 seconds; primary indicator of DoS attacks.

4. How to Explain This Project in Viva (Practical Q&A;)

Examiners frequently ask these foundational questions. Memorize these concise, high-scoring answers:

Q1: What is an Intrusion Detection System (IDS)?

An IDS is a cybersecurity defense tool that continuously monitors network traffic or system logs for malicious activity, policy violations, or unauthorized access attempts. When detected, it raises an alert for administrators.

Q2: Why use Machine Learning instead of traditional signature-based rules (like Snort)?

Traditional signature-based IDS only detects known attacks whose exact hashes or rules are stored in a database. If an attacker slightly modifies the payload (polymorphism or zero-day), signatures fail. Machine Learning learns generalized statistical patterns of legitimate vs. anomalous traffic, enabling it to detect new, unseen variants.

Q3: Why did you choose Random Forest Classifier?

Random Forest is an ensemble of 100 decision trees using bagging (bootstrap aggregation). It was chosen because: (1) It handles mixed categorical and continuous tabular network data exceptionally well, (2) Averaging multiple trees prevents overfitting, (3) It achieves 99%+ accuracy without heavy hyperparameter tuning, and (4) It provides interpretable feature importances.

Q4: What is the NSL-KDD dataset?

NSL-KDD was developed by the Canadian Institute for Cybersecurity (UNB) to solve critical flaws in the older KDD Cup 99 dataset. It eliminates millions of duplicate records that previously biased classifiers toward frequent packets, providing an unbiased benchmark for academic IDS.

Q5: What is the difference between Training and Testing data?

Training data (80% in our project) is used by the algorithm to build decision rules and learn internal node split thresholds. Testing data (20%) is strictly held-out data never seen during training, used exclusively to evaluate generalization performance.

Q6: What is a Confusion Matrix and what is the most dangerous error in IDS?

A Confusion Matrix is a 2x2 grid comparing actual labels against predictions (TN, FP, FN, TP). In cybersecurity, **False Negatives (FN)** are the most dangerous error because a missed attack penetrates the network, whereas False Positives (FP) merely cause an administrator to investigate a false alarm.

Q7: How does the Flask backend communicate with the Machine Learning model?

The trained Random Forest model and preprocessor are saved as binary files (.pkl) using Joblib. When the web interface submits JSON features via HTTP POST, Flask routes it to predict.py, runs the preprocessor (One-Hot + Scaler), calls model.predict() and model.predict_proba(), and returns the result (label, probability, risk level, explanation) as JSON back to the browser.

Q8: What is Overfitting and how did you prevent it?

Overfitting occurs when a model memorizes the training data and noise, failing on new data. We prevented overfitting by using an ensemble of 100 trees, constraining maximum tree depth (max_depth=15), and applying stratified train/test splitting.

5. Setup & Execution Commands

1. **Navigate:** `cd intrusion-detection-system`
2. **Install Dependencies:** `pip install -r requirements.txt`
3. **Train Model:** `python train_model.py`
4. **Start Web Application:** `python app.py`
5. **Open Browser:** `http://127.0.0.1:5000`
6. **Run Automated Tests:** `python test_app.py`